B.Bhanu Prasad

Batch no:38

2303a52021

Task 1: Sorting Student Records for Placement Drive

Scenario

SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.

Tasks

1. Use GitHub Copilot to generate a program that stores student records (Name, Roll Number, CGPA).

2. Implement the following sorting algorithms using AI assistance:

o Quick Sort

o Merge Sort

3. Measure and compare runtime performance for large datasets.

4. Write a function to display the top 10 students based on CGPA.

Expected Outcome

• Correctly sorted student records.

• Performance comparison between Quick Sort and Merge Sort.

• Clear output of top-performing students

AIAC_LAB 12.3.py > merge

```python
import time
import random
class Student:
    def __init__(self, name, roll_number, cgpa):
        self.name = name
        self.roll_number = roll_number
        self.cgpa = cgpa

    def __repr__(self):
        return f"Student({self.name}, {self.roll_number}, {self.cgpa})"

def quick_sort(students, low=0, high=None):
    if high is None:
        high = len(students) - 1

    if low < high:
        pi = partition(students, low, high)
        quick_sort(students, low, pi - 1)
        quick_sort(students, pi + 1, high)
    return students

def partition(students, low, high):
    pivot = students[high].cgpa
    i = low - 1
    for j in range(low, high):
        if students[j].cgpa > pivot:
            i += 1
            students[i], students[j] = students[j], students[i]
    students[i + 1], students[high] = students[high], students[i + 1]
    return i + 1

def merge_sort(students):
    if len(students) <= 1:
        return students

    mid = len(students) // 2
    left = merge_sort(students[:mid])
    right = merge_sort(students[mid:])
    return merge(left, right)

def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i].cgpa >= right[j].cgpa:
            result.append(left[i])
            i += 1
        else:
```

```python
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

def display_top_students(students, top_n=10):
    print(f"\nTop {top_n} Students:")
    print("-" * 60)
    print(f"{'Rank':<5} {'Name':<20} {'Roll Number':<15} {'CGPA':<10}")
    print("-" * 60)
    for i, student in enumerate(students[:top_n], 1):
        print(f"{i:<5} {student.name:<20} {student.roll_number:<15} {student.cgpa:<10.2f}")

def generate_sample_data(n=1000):
    names = ["Alice", "Bob", "Charlie", "Diana", "Eve", "Frank", "Grace", "Henry", "Iris", "Jack"]
    students = []
    for i in range(n):
        name = random.choice(names) + str(i)
        roll_number = f"SR{i+1:05d}"
        cgpa = round(random.uniform(6.0, 10.0), 2)
        students.append(Student(name, roll_number, cgpa))
    return students

if __name__ == "__main__":
    print("Student Records Sorting System")
    print("=" * 60)

    students = generate_sample_data(1000)

    # Quick Sort
    students_qs = [Student(s.name, s.roll_number, s.cgpa) for s in students]
    start_time = time.time()
    quick_sort(students_qs)
    qs_time = time.time() - start_time
    print(f"\nQuick Sort Time: {qs_time:.4f} seconds")

    # Merge Sort
    students_ms = [Student(s.name, s.roll_number, s.cgpa) for s in students]
    start_time = time.time()
    students_ms = merge_sort(students_ms)
    ms_time = time.time() - start_time
    print(f"Merge Sort Time: {ms_time:.4f} seconds")
```

```python
# Quick Sort
students_qs = [Student(s.name, s.roll_number, s.cgpa) for s in students]
start_time = time.time()
quick_sort(students_qs)
qs_time = time.time() - start_time
print(f"\nQuick Sort Time: {qs_time:.4f} seconds")

# Merge Sort
students_ms = [Student(s.name, s.roll_number, s.cgpa) for s in students]
start_time = time.time()
students_ms = merge_sort(students_ms)
ms_time = time.time() - start_time
print(f"Merge Sort Time: {ms_time:.4f} seconds")

# Performance Comparison
print(f"\n{'Performance Comparison':<30}")
print("-" * 40)
print(f"Quick Sort is {ms_time/qs_time:.2f}x faster" if qs_time < ms_time else f"Merge Sort is {qs_time/ms_time:.2f}x faster")

display_top_students(students_ms)
```

OUTPUT:

```python
display_top_students(students_ms)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
Quick Sort Time: 0.0009 seconds
Merge Sort Time: 0.0017 seconds

Performance Comparison
----------------------------------------
Quick Sort is 1.88x faster

Top 10 Students:
----------------------------------------------------------------
Rank  Name                 Roll Number     CGPA
----------------------------------------------------------------
1     Jack58               SR00059         10.00
2     Eve202               SR00203         9.99
3     Bob390               SR00391         9.99
4     Diana517             SR00518         9.99
5     Alice875             SR00876         9.99
6     Frank925             SR00926         9.99
7     Charlie235           SR00236         9.98
8     Jack307              SR00308         9.98
9     Diana990             SR00991         9.98
10    Eve595               SR00596         9.96
PS C:\Users\Bhanu Prasad\AIAC LAB>
```

Ln 45, Col 42 Spaces: 4 UTF-8 CRLF { } Python 3.14.2

EXPLANATION:

In the above code we are using both algorithm merge and quick sort foe storing the data of the student.The Quick sort is the faster than the merge sort.

Task 2: Implementing Bubble Sort with AI Comments

• Task: Write a Python implementation of Bubble Sort.

• Instructions:

• Students implement Bubble Sort normally.

• Ask AI to generate inline comments explaining key logic (like

swapping, passes, and termination).

• Request AI to provide time complexity analysis.

• Expected Output:

• A Bubble Sort implementation with AI-generated explanatory

comments and complexity analysis.

```python
def bubble_sort(arr):
    """
    Sorts an array using the Bubble Sort algorithm.

    Parameters:
    arr (list): The list to be sorted.

    Returns:
    list: The sorted list in ascending order.
    """
    n = len(arr)

    # Outer loop: Iterate through each pass
    # Each pass moves the largest unsorted element to its correct position
    for i in range(n):
        # Flag to optimize: if no swaps occur in a pass, array is already sorted
        swapped = False

        # Inner loop: Compare adjacent elements and swap if needed
        # Range decreases each iteration because last i elements are already sorted
        for j in range(0, n - i - 1):
            # Compare current element with next element
            if arr[j] > arr[j + 1]:
                # Swap: Move larger element towards the end
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        # Optimization: If no swaps occurred, array is sorted; terminate early
        if not swapped:
            break

    return arr


# Time Complexity Analysis:
# ──────────────────────────────────────────────────────────
# Best Case:    O(n)       - Array is already sorted
#               (Single pass with no swaps due to optimization flag)
#
# Average Case: O(n²)      - Random order array
#               (Multiple passes needed to sort elements)
#
# Worst Case:   O(n²)      - Array is sorted in reverse order
#               (Maximum comparisons and swaps required)
#
# Space Complexity: O(1)    - In-place sorting, only constant extra space
# ──────────────────────────────────────────────────────────
```

```python


# Example usage and test cases
if __name__ == "__main__":
    # Test case 1: Random array
    arr1 = [64, 34, 25, 12, 22, 11, 90]
    print("Original array:", arr1)
    print("Sorted array:", bubble_sort(arr1))

    # Test case 2: Already sorted array (demonstrates O(n) best case)
    arr2 = [1, 2, 3, 4, 5]
    print("\nAlready sorted array:", arr2)
    print("After bubble sort:", bubble_sort(arr2.copy()))

    # Test case 3: Reverse sorted array (demonstrates O(n²) worst case)
    arr3 = [5, 4, 3, 2, 1]
    print("\nReverse sorted array:", arr3)
    print("After bubble sort:", bubble_sort(arr3.copy()))

    # Test case 4: Array with duplicates
    arr4 = [3, 5, 3, 1, 5, 2]
    print("\nArray with duplicates:", arr4)
    print("After bubble sort:", bubble_sort(arr4.copy()))
```

OUTPUT:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                                   Python: AIAC_LAB 12.3

> & C:/Python314/python.exe "c:/Users/Bhanu Prasad/AIAC LAB/AIAC_LAB 12.3.py"
Original array: [64, 34, 25, 12, 22, 11, 90]
Sorted array: [11, 12, 22, 25, 34, 64, 90]

Already sorted array: [1, 2, 3, 4, 5]
After bubble sort: [1, 2, 3, 4, 5]

Reverse sorted array: [5, 4, 3, 2, 1]
After bubble sort: [1, 2, 3, 4, 5]

Array with duplicates: [3, 5, 3, 1, 5, 2]
After bubble sort: [1, 2, 3, 3, 5, 5]
PS C:\Users\Bhanu Prasad\AIAC LAB>
```

EXPLANATION:
We are performing the bubble sort in the above code.The output is in ascending order. When we give the random input it is sorting them in ascending order.

Task 3: Quick Sort and Merge Sort Comparison
• Task: Implement Quick Sort and Merge Sort using recursion.
• Instructions:
• Provide AI with partially completed functions for recursion.
• Ask AI to complete the missing logic and add docstrings.
• Compare both algorithms on random, sorted, and reverse-sorted lists.
• Expected Output:
• Working Quick Sort and Merge Sort implementations.
• AI-generated explanation of average, best, and worst-case complexities.

```python
'''
import time
import random
class Student:
    def __init__(self, name, roll_number, cgpa):
        self.name = name
        self.roll_number = roll_number
        self.cgpa = cgpa

    def __repr__(self):
        return f"Student({self.name}, {self.roll_number}, {self.cgpa})"

def quick_sort(students, low=0, high=None):
    """
    Sorts students by CGPA in descending order using Quick Sort.

    Time Complexity:
    - Best/Average Case: O(n log n)
    - Worst Case: O(n²) when pivot is always smallest/largest

    Space Complexity: O(log n) - recursive call stack
    """
    if high is None:
        high = len(students) - 1

    if low < high:
        pi = partition(students, low, high)
        quick_sort(students, low, pi - 1)
        quick_sort(students, pi + 1, high)
    return students

def partition(students, low, high):
    """Partitions array around pivot element."""
    pivot = students[high].cgpa
    i = low - 1
    for j in range(low, high):
        if students[j].cgpa > pivot:
            i += 1
            students[i], students[j] = students[j], students[i]
    students[i + 1], students[high] = students[high], students[i + 1]
    return i + 1

def merge_sort(students):
```

```python
def partition(students, low, high):
    """Partitions array around pivot element."""
    pivot = students[high].cgpa
    i = low - 1
    for j in range(low, high):
        if students[j].cgpa > pivot:
            i += 1
            students[i], students[j] = students[j], students[i]
    students[i + 1], students[high] = students[high], students[i + 1]
    return i + 1

def merge_sort(students):
    """
    Sorts students by CGPA in descending order using Merge Sort.

    Time Complexity:
    - Best/Average/Worst Case: O(n log n)
    - Guaranteed O(n log n) performance

    Space Complexity: O(n) - requires additional space for merging
    """
    if len(students) <= 1:
        return students

    mid = len(students) // 2
    left = merge_sort(students[:mid])
    right = merge_sort(students[mid:])
    return merge(left, right)

def merge(left, right):
    """Merges two sorted lists in descending order."""
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i].cgpa >= right[j].cgpa:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
```

```python
def merge(left, right):
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

def display_top_students(students, top_n=10):
    """Displays top N students sorted by CGPA."""
    print(f"\nTop {top_n} Students:")
    print("-" * 60)
    print(f"{'Rank':<5} {'Name':<20} {'Roll Number':<15} {'CGPA':<10}")
    print("-" * 60)
    for i, student in enumerate(students[:top_n], 1):
        print(f"{i:<5} {student.name:<20} {student.roll_number:<15} {student.cgpa:<10.2f}")

def generate_sample_data(n=1000):
    """Generates random student records."""
    names = ["Alice", "Bob", "Charlie", "Diana", "Eve", "Frank", "Grace", "Henry", "Iris", "Jack"]
    students = []
    for i in range(n):
        name = random.choice(names) + str(i)
        roll_number = f"SR{i+1:05d}"
        cgpa = round(random.uniform(6.0, 10.0), 2)
        students.append(Student(name, roll_number, cgpa))
    return students

if __name__ == "__main__":
    print("Quick Sort vs Merge Sort Comparison")
    print("=" * 60)

    students = generate_sample_data(1000)

    # Quick Sort
    students_qs = [Student(s.name, s.roll_number, s.cgpa) for s in students]
    start_time = time.time()
    quick_sort(students_qs)
    qs_time = time.time() - start_time
    print(f"\nQuick Sort Time: {qs_time:.4f} seconds")

    # Merge Sort
    students_ms = [Student(s.name, s.roll_number, s.cgpa) for s in students]
    start_time = time.time()
    students_ms = merge_sort(students_ms)
    ms_time = time.time() - start_time
    print(f"Merge Sort Time: {ms_time:.4f} seconds")

    # Performance Comparison
    print(f"\n{'Performance Comparison':<30}")
    print("-" * 40)
    print(f"Quick Sort is {ms_time/qs_time:.2f}x faster" if qs_time < ms_time else f"Merge Sort is {qs_time/ms_time:.2f}x faster")

    display_top_students(students_ms)
```

OUTPUT:

```
Quick Sort vs Merge Sort Comparison
============================================================

Quick Sort Time: 0.0019 seconds
Merge Sort Time: 0.0050 seconds

Performance Comparison
----------------------------------------
Quick Sort is 2.64x faster

Top 10 Students:
------------------------------------------------------------
Rank  Name                 Roll Number     CGPA
------------------------------------------------------------
1     Grace649             SR00650         10.00
2     Grace082             SR00083         10.00
3     Iris240              SR00241         9.99
4     Grace452             SR00453         9.99
5     Iris525              SR00526         9.99
6     Iris602              SR00603         9.99
7     Jack701              SR00702         9.99
8     Charlie775           SR00776         9.99
9     Frank6               SR00007         9.98
10    Bob698               SR00699         9.98
PS C:\Users\Bhanu Prasad\AIAC LAB>
```

EXPLANATION.

We are comparing the time complexity of merge and quick.quick sort is faster then the merge sort.

Task 4 (Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.
2. Sort products by price or quantity for stock analysis.

Task:

• Use AI to suggest the most efficient search and sort algorithms for this use case.

• Implement the recommended algorithms in Python.

• Justify the choice based on dataset size, update frequency, and performance requirements.

Expected Output:

• A table mapping operation → recommended algorithm → justification.

• Working Python functions for searching and sorting the inventory.

```python
class Product:
    def __init__(self, product_id, name, price, stock):
        self.product_id = product_id
        self.name = name
        self.price = price
        self.stock = stock

    def __repr__(self):
        return f"Product({self.product_id}, {self.name}, $({self.price}, Stock: {self.stock})"

# ============================================================================
# SEARCH ALGORITHMS
# ============================================================================

def linear_search(products, key, search_by='id'):
    """
    Linear Search - O(n)
    Best for: Small datasets, unsorted data, infrequent searches
    """
    for product in products:
        if search_by == 'id' and product.product_id == key:
            return product
        elif search_by == 'name' and product.name.lower() == key.lower():
            return product
    return None

def binary_search(products, key, search_by='id'):
    """
    Binary Search - O(log n)
    Requires: Pre-sorted list
    Best for: Large datasets, frequent searches
    """
    left, right = 0, len(products) - 1

    while left <= right:
        mid = (left + right) // 2
        if search_by == 'id':
            mid_val = products[mid].product_id
        else:
            mid_val = products[mid].name.lower()

        if mid_val == (key if search_by == 'id' else key.lower()):
            return products[mid]
        elif mid_val < key:
            left = mid + 1
```

```python
def hash_table_search(products, search_by='id'):
    """
    Hash Table - O(1) average case
    Best for: Very frequent searches, large datasets
    Pre-process products into dictionary for O(1) lookup
    """
    hash_map = {}
    if search_by == 'id':
        for product in products:
            hash_map[product.product_id] = product
    else:
        for product in products:
            hash_map[product.name.lower()] = product
    return hash_map

# ============================================================================
# SORT ALGORITHMS
# ============================================================================

def quick_sort_products(products, low=0, high=None, sort_by='price'):
    """
    Quick Sort - O(n log n) average, O(n²) worst
    Best for: General purpose, in-place sorting, medium to large datasets
    """
    if high is None:
        high = len(products) - 1

    if low < high:
        pi = partition_products(products, low, high, sort_by)
        quick_sort_products(products, low, pi - 1, sort_by)
        quick_sort_products(products, pi + 1, high, sort_by)
    return products

def partition_products(products, low, high, sort_by):
    pivot_val = products[high].price if sort_by == 'price' else products[high].stock
    i = low - 1
    for j in range(low, high):
        current_val = products[j].price if sort_by == 'price' else products[j].stock
        if current_val < pivot_val:
            i += 1
            products[i], products[j] = products[j], products[i]
    products[i + 1], products[high] = products[high], products[i + 1]
    return i + 1

def merge_sort_products(products, sort_by='price'):
    """
    Merge Sort - O(n log n) guaranteed
    Best for: Consistent performance, stable sort, large datasets
    Trade-off: Requires O(n) extra space
    """
    if len(products) <= 1:
        return products

    mid = len(products) // 2
    left = merge_sort_products(products[:mid], sort_by)
    right = merge_sort_products(products[mid:], sort_by)
    return merge_products(left, right, sort_by)
```

```python
def merge_products(left, right, sort_by):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        left_val = left[i].price if sort_by == 'price' else left[i].stock
        right_val = right[j].price if sort_by == 'price' else right[j].stock
        if left_val <= right_val:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1
    result.extend(left[i:])
    result.extend(right[j:])
    return result

# ============================================================================
# ALGORITHM RECOMMENDATION TABLE
# ============================================================================

def print_recommendations():
    print("\n" + "=" * 100)
    print("ALGORITHM RECOMMENDATIONS FOR INVENTORY MANAGEMENT")
    print("=" * 100)
    print(f"{'Operation':<20} {'Recommended':<20} {'Time Complexity':<25} {'Justification':<35}")
    print("-" * 100)

    recommendations = [
        ("Search by ID", "Hash Table", "O(1) average", "Frequent searches, instant lookup"),
        ("Search by Name", "Hash Table", "O(1) average", "Customer-facing, needs speed"),
        ("Sort by Price", "Merge Sort", "O(n log n)", "Guaranteed performance, stable"),
        ("Sort by Stock", "Quick Sort", "O(n log n) avg", "Good cache locality, fast"),
        ("One-time Search", "Binary Search", "O(log n)", "If pre-sorted, low memory"),
    ]

    for op, algo, complexity, justification in recommendations:
        print(f"{op:<20} {algo:<20} {complexity:<25} {justification:<35}")
    print("=" * 100)

# ============================================================================
# PERFORMANCE TESTING
# ============================================================================

def generate_inventory(n=10000):
    """Generate random product inventory."""
    product_names = ["Laptop", "Mouse", "Keyboard", "Monitor", "Headphones", "USB Cable", "Charger", "Screen Protector"]
    products = []
    for i in range(n):
        products.append(Product(
            product_id=f"P{i+1:06d}",
            name=random.choice(product_names) + f" {i}",
            price=round(random.uniform(10, 1000), 2),
            stock=random.randint(0, 500)
        ))
    return products
```

```python
    return products

if __name__ == "__main__":
    print("\nINVENTORY MANAGEMENT SYSTEM - Algorithm Analysis")
    print("=" * 100)

    products = generate_inventory(10000)

    # Display recommendations
    print_recommendations()

    # Test Hash Table Search (Most Efficient)
    print("\n1. HASH TABLE SEARCH - O(1) Average Case")
    print("-" * 100)
    hash_map = hash_table_search(products, 'id')
    start = time.time()
    for _ in range(1000):
        hash_map.get("P000500")
    hash_time = time.time() - start
    print(f"Hash Table Search (1000 lookups): {hash_time:.6f} seconds")

    # Test Quick Sort
    print("\n2. QUICK SORT - O(n log n) Average Case")
    print("-" * 100)
    products_qs = [Product(p.product_id, p.name, p.price, p.stock) for p in products]
    start = time.time()
    quick_sort_products(products_qs, sort_by='price')
    qs_time = time.time() - start
    print(f"Quick Sort (10000 items by price): {qs_time:.4f} seconds")
    print(f"Top 5 Most Expensive: {products_qs[-5:]}")

    # Test Merge Sort
    print("\n3. MERGE SORT - O(n log n) Guaranteed")
    print("-" * 100)
    products_ms = [Product(p.product_id, p.name, p.price, p.stock) for p in products]
    start = time.time()
    products_ms = merge_sort_products(products_ms, sort_by='stock')
    ms_time = time.time() - start
    print(f"Merge Sort (10000 items by stock): {ms_time:.4f} seconds")
    print(f"Top 5 Overstocked: {products_ms[-5:]}")
```

OUTPUT:

```
466     # Test Hash Table Search (Most Efficient)

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS                                   Python: AIAC_LAB 12.3

PS C:\Users\Bhanu Prasad\AIAC LAB> & C:/Python314/python.exe "c:/Users/Bhanu Prasad/AIAC LAB/AIAC_LAB 12.3.py"

INVENTORY MANAGEMENT SYSTEM - Algorithm Analysis
====================================================================================================

====================================================================================================
ALGORITHM RECOMMENDATIONS FOR INVENTORY MANAGEMENT
====================================================================================================
Operation            Recommended          Time Complexity          Justification
----------------------------------------------------------------------------------------------------
Search by ID         Hash Table           O(1) average            Frequent searches, instant lookup
Search by Name       Hash Table           O(1) average            Customer-facing, needs speed
Sort by Price        Merge Sort           O(n log n)              Guaranteed performance, stable
Sort by Stock        Quick Sort           O(n log n) avg          Good cache locality, fast
One-time Search      Binary Search        O(log n)                If pre-sorted, low memory
====================================================================================================

1. HASH TABLE SEARCH - O(1) Average Case
----------------------------------------------------------------------------------------------------
Hash Table Search (1000 lookups): 0.000134 seconds

2. QUICK SORT - O(n log n) Average Case
----------------------------------------------------------------------------------------------------
Quick Sort (10000 items by price): 0.0415 seconds
Top 5 Most Expensive: [Product(P009494, Mouse 9493, $999.63, Stock: 392), Product(P008920, Keyboard 8919, $999.86, Stock: 279), Product(P005769, Headphones 5768, $999.87, Stock: 9), Product(P004174, Keyboard 4173, $999.94, Stock: 282), Product(P007072, Charger 707
1, $999.94, Stock: 102)]

3. MERGE SORT - O(n log n) Guaranteed
----------------------------------------------------------------------------------------------------
Merge Sort (10000 items by stock): 0.0615 seconds
Top 5 Overstocked: [Product(P005368, Headphones 5367, $425.69, Stock: 500), Product(P006069, USB Cable 6068, $544.34, Stock: 500), Product(P007301, Monitor 7300, $212.28, Stock: 500), Product(P009117, Keyboard 9116, $750.34, Stock: 500), Product(P009762, Keyboard
9761, $395.46, Stock: 500)]
PS C:\Users\Bhanu Prasad\AIAC LAB>
```

EXPLANATION:

The output shows algorithm recommendations for inventory operations and their efficiency. Hash tables give instant searches, while Quick Sort and Merge Sort handle large dataset sorting quickly with sample product results.

TASK :5:

Real-Time Stock Data Sorting & Searching

Scenario:

An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.

- Use GitHub Copilot to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).
- Implement sorting algorithms to rank stocks by percentage change.
- Implement a search function that retrieves stock data instantly when a stock symbol is entered.

• Optimize sorting with Heap Sort and searching with Hash Maps.
• Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs.

```python
import time
import random
class Stock:
    def __init__(self, symbol, open_price, close_price):
        self.symbol = symbol
        self.open_price = open_price
        self.close_price = close_price
        self.change = close_price - open_price
        self.percent_change = (self.change / open_price) * 100

    def __repr__(self):
        return f"Stock({self.symbol}, {self.open_price}, {self.close_price}, {self.percent_change:.2f}%)"

def heap_sort(stocks):
    """Heap Sort - O(n log n) guaranteed, in-place"""
    def heapify(arr, n, i):
        largest = i
        left = 2 * i + 1
        right = 2 * i + 2

        if left < n and arr[left].percent_change > arr[largest].percent_change:
            largest = left
        if right < n and arr[right].percent_change > arr[largest].percent_change:
            largest = right

        if largest != i:
            arr[i], arr[largest] = arr[largest], arr[i]
            heapify(arr, n, largest)

    n = len(stocks)
    for i in range(n // 2 - 1, -1, -1):
        heapify(stocks, n, i)

    for i in range(n - 1, 0, -1):
        stocks[0], stocks[i] = stocks[i], stocks[0]
        heapify(stocks, i, 0)

    return stocks

def hash_map_search(stocks):
    """Create hash map for O(1) stock symbol lookup"""
    return {stock.symbol: stock for stock in stocks}

def generate_stock_data(n=1000):
    """Generate random stock data"""
    symbols = ["AAPL", "GOOGL", "MSFT", "TSLA", "AMZN", "META", "NFLX", "NVDA", "AMD", "INTEL"]
    stocks = []
    for i in range(n):
        symbol = random.choice(symbols) + str(i % 100)
        open_price = round(random.uniform(50, 500), 2)
        close_price = round(open_price * random.uniform(0.95, 1.05), 2)
        stocks.append(Stock(symbol, open_price, close_price))
    return stocks

if __name__ == "__main__":
    print("Real-Time Stock Data Sorting & Searching System")
    print("=" * 80)

    stocks = generate_stock_data(1000)

    # Heap Sort
    print("\n1. HEAP SORT - O(n log n) In-place")
    print("-" * 80)
    stocks_heap = [Stock(s.symbol, s.open_price, s.close_price) for s in stocks]
    start = time.time()
    heap_sort(stocks_heap)
    heap_time = time.time() - start
    print(f"Heap Sort Time: {heap_time:.4f} seconds")
    print(f"Top 5 Gainers:\n{chr(10).join([f'{i+1}. {s}' for i, s in enumerate(stocks_heap[:5])])}")

    # Standard Library Sort
    print("\n2. STANDARD LIBRARY SORTED() - O(n log n) Timsort")
    print("-" * 80)
    stocks_std = [Stock(s.symbol, s.open_price, s.close_price) for s in stocks]
    start = time.time()
    stocks_std = sorted(stocks_std, key=lambda x: x.percent_change, reverse=True)
    std_time = time.time() - start
    print(f"Sorted() Time: {std_time:.4f} seconds")

    # Hash Map Search
    print("\n3. HASH MAP SEARCH - O(1) Average")
    print("-" * 80)
    stock_map = hash_map_search(stocks)
    search_symbols = [stocks[i].symbol for i in range(0, min(100, len(stocks)), 10)]
    start = time.time()
    for sym in search_symbols * 10:
        stock_map.get(sym)
    hash_time = time.time() - start
    print(f"Hash Map Search (1000 lookups): {hash_time:.6f} seconds")

    # Linear Search Comparison
    print("\n4. LINEAR SEARCH - O(n)")
    print("-" * 80)
    start = time.time()
    for sym in search_symbols * 10:
        next((s for s in stocks if s.symbol == sym), None)
    linear_time = time.time() - start
    print(f"Linear Search (1000 lookups): {linear_time:.6f} seconds")

    # Performance Comparison
    print("\n5. PERFORMANCE COMPARISON")
```

Ln 574, Col 35 Spaces: 4 UTF-

OUTPUT:

```
Linear Search (1000 lookups): 0.000554 seconds

5. PERFORMANCE COMPARISON
--------------------------------------------------------------------------------
Hash Map vs Linear Search: 21.14x faster
Heap Sort vs Sorted(): 0.06x slower
PS C:\Users\Bhanu Prasad\AIAC LAB>
```

EXPLANATION:

In the above code we are comparing the heapmap vs linear search.Heap is faster when compared to both sorted and linear.